



DataLake 수도꼭지 — 사용자 매뉴얼

turn the valve · feed the flow

이 문서는 Valve 를 사내에 배포하고 운영·확장하려는 엔지니어를 위한 실무 매뉴얼입니다. 17개 섹션에 걸쳐 설정 · 사용법 · 알림 · 에이전트 연동 · flow 와의 통합까지 다룹니다.

페어 앱: [flow](https://github.com/goood2280/flow) (github.com/goood2280/flow)

저장소: github.com/goood2280/valve (private)

버전: v0.1.1 (2026-04-24)

1. Valve 는 무엇인가

Valve 는 사내 DataLake API 에서 데이터를 꺼내 parquet staging + S3 업로드 까지 흘러 flow 앱에 공급하는 수도꼭지입니다. 사내 실무 환경의 3가지 고통을 해결합니다.

- ① 쿼리 함수 호출 불안정성. HY000/ODBC 에러 5% · 5분 timeout · 동시 실행 제약 — Valve 는 retry · rate-limit · chunk 분할로 흡수합니다.
- ② 하루치 데이터가 큼. probe 로 분포 먼저 스캔 → shard 로 나눠 병렬 추출 → timeout 시 자동 re-shard.
- ③ 여러 제품·여러 소스의 관리 피로. products.yaml / source_types.yaml 로 제품·DB 유형·추출 컬럼을 편집 가능. 웹 UI 에서 한눈에.

핵심 설계 원칙: Robust > Smart. 모든 실패는 기록되고, 알림은 3-채널로 fan-out 되며, 설정이 손상되면 직전 정상값으로 fallback — 서비스가 중단되지 않습니다.

2. Quick Start (5분 안에 돌려보기)

1) 설치

```
git clone https://github.com/goood2280/valve
cd valve
pip install -r requirements.txt
```

2) 실행

```
python -m uvicorn app:app --host 0.0.0.0 --port 8090
```

3) 브라우저 열기: http://localhost:8090

4) 모니터 탭에서 [▶ 전체 실행] 눌러 mock 데이터로 PRODA/PRODB 가 FAB/INLINE/ET 로 뽑혀 staging 에 쌓이는지 확인.

5) smoke test

```
python scripts/smoke_test.py http://127.0.0.1:8090
# 41      <0.5s
```

기본은 mock 모드. 실 사내 API 연결은 Settings > Lake API 에서 mode=real + module 문자열 지정.

3. 아키텍처 한눈에

```
[ DataLake API ]
| lake_api.py retry · timeout · rate-limit
▼
[Planner] probe(sample_window|projection|none) → shard    → chunk plan
|
▼
[Executor]  chunk    ·    →reshard · staging
|
|— staging/{product}/{source}/date=YYYY-MM-DD/*.parquet
|
▼
```

```
[S3 Queue] upload_mode = immediate | interval | manual
|
▼
s3://bucket/{prefix}/{source}/{product}/date=.../part-0.parquet
|
▼
[flow app] SplitTable · Dashboard · ML · ...
```

상태는 StateStore (jobs.jsonl append-only) 에 기록되고 SSE 스트림 으로 Monitor UI 에 실시간 반영됩니다.

4. Settings > 사내 Lake API

실제 데이터를 가져오는 함수 호출을 어떻게 할지 정합니다.

필드	의미 · 권장값
mode	mock (데모/개발) 또는 real. real 로 바꾸면 아래 module 로드.
module	mycorp.datalake:query 형태. importlib 로 동적 로드.
user	사내 query 함수의 user 인자.
api_key	사내 API 인증 키. 있을 때만 query(..., api_key=...) kwarg 로 전달. 저장 시 **** 마스크.
timeout_sec	5분(300) 이하 권장. 기본 290.
min_interval_sec	rate-limit — 두 호출 사이 최소 간격(초).
max_concurrent	동시 chunk 수. 1~5 권장.
retry.attempts	실패 시 최대 재시도 횟수.
retry.backoff_sec	각 재시도 전 지연. [10,30,120] 같은 리스트.
retryable_errors	재시도 트리거 문자열. HY000, TimeoutError 등.

실 모드 팁: query 함수가 api_key kwarg 를 안 받아도 TypeError 감지 후 3-인자 폴백하므로 구버전 어댑터 그대로 호환됩니다.

5. Settings > S3 업로드

staging 에 쌓인 parquet 을 S3 로 언제 어떻게 올릴지 정합니다. upload_mode 3종을 이해하는 게 핵심.

필드	의미
endpoint_url	비우면 AWS S3. MinIO 는 http://host:9000 형태.
bucket / prefix / access_key / secret_key	표준 boto3 파라미터. secret_key 는 저장 후 ****.
fake_local_path	endpoint_url 비우고 이 값 있으면 개발 모드 — 로컬 폴더에 S3 흉내 쓰기.
upload_mode	immediate (기본) interval manual
upload_interval_sec	interval 모드에서만 의미. 기본 300초(5분). 최소 5초 강제.
retry_failed_sec	업로드 실패 항목 재시도 간격. 기본 120초. 3회 실패 시 알람.

업로드 모드 선택 가이드

모드	언제 쓰나	특징
immediate	일반 운영. 소규모~중규모.	chunk 완료 시 바로 put_atomic. 지연 적음, 실패 즉시 status=upload
interval	대량 chunk 집중 발생 구간 (초기 시딩 중).	큐잉 후 주기로 flush. S3 부하 평탄화. 실패는 자동 retry.
manual	엔지니어가 S3 비용/타이밍 직접 제어.	큐잉만. /api/jobs/s3-flush 눌러야 실제 올라감. CI/cron 통합 쉬움.

6. Settings > 스케줄 · 프로브

스케줄

필드	의미
backfill_days	오늘 + 과거 N일을 backfill 창으로 사용. 3~5 권장. 제품별 override 는 제품 탭의 backfill 필드.
interval_hours	자동 스케줄 주기 (v0.2 예정 — 현재 수동 /enqueue-all).
force_overwrite	기존 파티션을 덮어쓸지. true 면 재실행 시 replace.
tolerance_pct	completeness 허용 오차 %. 0.5 = 실제/예상 차이 0.5% 이내면 OK.

프로브 (분포 미리보기)

필드	의미
strategy	sample_window — 1시간 샘플 조회로 24배 비례 추정. projection — 하루치 전체지만 shard 컬럼만 받아 정확 분포. none — probe 생략, 단일 chunk 로 바로 실행.
window_hours	sample_window 샘플 범위. 1시간 기본.
cache_days	같은 (제품,소스) probe 결과 N일 재사용. 기본 7.
adaptive_correction	probe 추정이 틀렸을 때 chunk 실행 중 감지·재분할.
fallback_on_timeout	probe 자체가 timeout 나면 none 으로 즉시 진행.

probe 가 자주 실패하는 소스는 제품 탭의 소스 설정에 probe skip 체크박스 ON — 해당 소스만 probe 없이 단일 chunk 로 바로 실행.

7. Settings > 알람 (3-채널 fan-out)

이상이 발생하면 S3 업로드 · flow 앱 푸시 · 범용 webhook 3채널로 동시 전송합니다. 각 채널은 독립 on/off, rate-limit + dedupe 로 폭주 차단.

필드	의미
enabled	마스터 스위치. 끄면 모든 채널 무시.
min_severity	info < warn < error < critical. 이 레벨 미만은 drop.
max_per_hour	시간당 최대 알람 수 (sliding window). 0 이면 무제한.
dedupe_window_sec	같은 (kind + chunk_id) 에 대해 이 시간 내 중복 억제. 0 이면 없음.
s3_enabled / s3_prefix	S3 에 알람 JSON 누적. prefix 기본 valve-alerts. s3://bucket/valve-alerts/20260424T121314-chun
flow_enabled / flow_notify_url	flow 앱 알람 엔드포인트. Valve↔flow 연동 기본 채널.
webhook_enabled / webhook_url	범용 webhook. Slack/Teams 어댑터 등.
config_prefix	Valve 기동 시 S3 에서 settings/products/source_types 를 pull 할 prefix. 기본 valve-config.

알람 이벤트 페이로드

```
{
  "ts": 1776991200.0,
  "source": "valve.executor",
  "kind": "chunk_failed",
  "severity": "error",
  "title": "PRODA/FAB/2026-04-24 chunk failed",
  "chunk_id": "PRODA-FAB-2026-04-24-00",
  "product": "PRODA", "source": "FAB", "date": "2026-04-24",
  "error_type": "HY000Error",
  "error": "[HY000] ODBC driver error",
  "dispatch": {
    "s3": {"ok": true, "key": "valve-alerts/20260424T120000-chunk_failed.json"},
    "flow": {"ok": true, "message": "200 "},
    "webhook": null
  }
}
```

GET /api/alerts/recent?limit=50 로 최근 200건 메모리 버퍼 조회 가능.

8. Settings > 소스 타입 (동적 레지스트리)

신규 DB 유형이 생기면 여기서 한 줄 추가 → 즉시 Products 편집기 드롭다운에 노출.

내장 6종: FAB · INLINE · ET · QTIME · EDS · VM. 각 소스 마다 컬럼 풀 · 기본 shard · 가이드 힌트 · 강조 색상이 지정됩니다.

필드	의미
name	대문자 유일 키. 파티션 폴더/S3 prefix 에 그대로 쓰임.
table_template	기본 테이블명. {name} 토큰은 name 으로 치환.
columns	이 소스에서 추출 가능한 컬럼 풀 (제품 편집기 드롭다운).
default_shard	신규 소스 추가 시 제안되는 기본 shard_hierarchy.

필드	의미
accent	hint 박스와 히트맵 테두리 색상 (HEX).
hint	가이드 문구. `` 로 감싸면 inline code 렌더.

저장하면 config/source_types.yaml 로 기록되고 /api/schedule/source-types GET/POST 로도 프로그램 접근 가능.

9. Products 탭 — 제품 · DB · 필터의 모든 것

좌측 제품 목록 · 우측 상세 split. 제품 클릭으로 포커스, 내부 소스 탭으로 drill-down. 각 소스 마지막에 최종 Python 호출 미리보기.

제품 헤더

요소	용도
name 입력	PRODA/PRODB 식 이름.
enabled 체크	끄면 스케줄에서 제외.
priority	낮을수록 우선. enqueue-all 순서 정렬.
backfill	제품별 backfill 일수 override. 비우면 전역값. 신규 세팅 시 300-600 등 크게 → 시딩 끝나면 비우기.
초기 시딩	이 제품만 backfill 전 기간 × 모든 소스 일괄 투입.
제품 삭제	products.yaml 에서 제거. 저장 눌러야 실제 반영.

제품 공통 기본 (1급 필드)

process_id · line_id · product_code 세 가지는 제품 고유 식별자이므로 별도 그리드로 승격. 심표로 여러 개 입력 시 자동으로 IN 연산자.

```
params_template = {
  "process_id": {"op": "in", "value": ["P4203", "P4204"]},
  "line_id": {"op": "eq", "value": "L01"},
  "product_code": {"op": "eq", "value": "PRODA"},
}
```

추가 필터

그 외 WHERE 조건. like / notLike 연산자 지원 (%AA% 패턴).

```
# cata      '%AA%'
params_template["cata"] = {"op": "notLike", "value": "%AA%"}
```

📖 추출 소스 + 최종 호출 미리보기

소스별 탭 (FAB/INLINE/ET/...) 에 버튼으로 삭제. + 버튼으로 추가 (SOURCE_NAMES 6종 우선, 6종 초과 시 NEW).

각 소스 카드 하단의 이 소스의 최종 호출 (Python):

```
df: pandas.DataFrame = query(
  params={
    "table": "RAW_FAB_DATA",
    "dateFrom": "2026-04-24T00:00:00",
    "dateTo": "2026-04-25T00:00:00",
    "process_id": {"op": "in", "value": ["P4203", "P4204"]},
    "product_code": {"op": "eq", "value": "PRODA"},
  },
  custom_col=["lot_id", "wafer_id", "time", "item_id", "value"],
  user="pipe-runner",
  api_key="{settings.lake_api.api_key}",
)
```


10. Monitor 히트맵 읽는 법

14일치 (기본) 의 (제품 × 소스 × 날짜) 상태를 한 화면에.

색	상태
녹색 (Success)	정상 완료. 완전성·업로드 OK.
파랑 (Running)	현재 chunk 실행 중. 펄스 애니메이션.
노랑 (Partial / Tolerance)	일부 chunk 실패했거나 완전성 초과. 재실행 고려.
빨강 (Failed / Upload err)	치명. Logs 탭에서 원인 확인.
회색 (Planned)	스케줄은 있으나 아직 실행 안 됨.
투명 (Idle)	해당 날짜 소스 추출 계획 없음.
대각선 빗금 (미추출)	이 제품은 해당 소스를 아예 추출하지 않음 (products.yaml 기준).

제품 그룹 헤더 에는 priority · backfill override · (추출 중 / 전체 소스 수) 가 요약. 각 셀 클릭 시 해당 (제품,소스,날짜) 재실행 투입 확인 다이얼로그.

11. Logs 탭 — 언제 시도했고 왜 실패했는지

각 chunk / plan / partition 의 마지막 상태 기록. 제품 · 소스 · 상태 · 실패만 · 종류 (chunk/plan/partition/all) 필터. 15초 자동 새로고침.

행 색	의미
빨강 배경	failed / timeout_reshard / completeness_failed / upload_failed.
노랑 배경	plan 단계에서 probe 실패 → 단일 chunk fallback.
흰색	정상.

백엔드 소스: GET /api/jobs/history?limit=300&failed_only=true&kind=chunk. jobs.jsonl 뒤에서부터 tail. 같은 chunk_id 의 최신 상태만 1회 반환.

12. Browser 탭 — staging/s3_local parquet 검사

추출된 parquet 을 좌측 트리로 탐색, 선택한 파일을 polars SQLContext 로 실시간 필터링.

상단 SQL 사용 가이드 접기 패널에 10가지 예시 snippet (클릭 시 적용).

```
-- :
-- 1.      t. SELECT * FROM t WHERE ...
-- 2. FROM      SELECT * FROM t WHERE ...
-- 3.      'single-quote' (backtick/double-quote )
-- 4.      2000 .      LIMIT
-- 5.      ISO      CAST(... AS TIMESTAMP)

-- :
SELECT lot_id, wafer_id, time, value
FROM t
WHERE item_id = 'ITEM_042'
ORDER BY time DESC
LIMIT 100
```

13. S3 Config Sync — 설정을 중앙에서

여러 Valve 인스턴스를 같은 설정으로 돌리려면 S3 의 valve-config/ 에 settings.json · products.yaml · source_types.yaml 을 올려두고 각 인스턴스가 기동 시 pull 하게 합니다.

```
s3://bucket/valve-config/settings.json
s3://bucket/valve-config/products.yaml
s3://bucket/valve-config/source_types.yaml
```

4-단계 Fallback 체인 (robust > fast)

#	상태	동작
1	S3 정상 + 파서 통과	로컬 파일 교체 + .last_good 백업. 알람 없음.
2	S3 파일 파싱 실패	config_s3_invalid 경고 발행, 로컬 파일로 fallback.
3	S3 미도달 (네트워크/권한)	config_s3_unreachable 경고 발행, 로컬 파일로 fallback.
4	로컬 파일마저 손상	config_local_corrupt 에러 발행, .last_good 로 복구.
5	전부 사용 불가	config_missing 에러 발행, 번들 기본값으로 시작.

각 단계 전환은 3-채널 알람(S3-flow-webhook) 으로 동시 브로드캐스트. 운영자는 채널 하나만 보고 있어도 인지 가능.

14. Agent 연동 — 오픈소스 LLM 친화 설계

/api/agent/* 네임스페이스로 진단·제안·실행 3-tier 제공. LLM 은 '읽고 고르고 호출' 만 함 — 창작 금지.

엔드포인트	역할
GET /api/agent/diagnose	현재 이상 목록 (chunk_failed · stuck · probe_error · partition_partial). 서버가 규칙 기반으로
POST /api/agent/suggest-fix	이상 1건을 받아 후보 fix 반환 — action · args · confidence · rationale · safety(LOW/MEDIUM)
POST /api/agent/apply-fix	액션 실행. dry_run: true 기본 — 검증만. false 로 전환해야 실 적용.
GET /api/agent/actions	화이트리스트 액션 카탈로그 (retry_chunk · reshard_source · toggle_probe_skip 등 8종).
GET /api/agent/audit	모든 agent 호출 감사 로그.

안전 장치 5중

- 화이트리스트 외 액션은 400 Bad Request.
- 인자 누락 시 필요 인자 목록 반환.
- dry_run 우선 — 검증 실패하면 real 차단.
- 60초 cooldown per (action + args).
- 동일 키에 3연속 실패하면 suspend — 관리자 해제 필요.

자세한 설계는 docs/agent_design.md 참고.

15. flow 앱과 연동

flow 는 Valve 가 쓴 parquet 을 그대로 읽어 SplitTable/Dashboard/ML 에 활용합니다.
포맷·파티셔닝·스키마는 이미 일치 — prefix 1줄만 정렬하면 끝.

관점	일치 여부
parquet 포맷 · hive date= 파티셔닝	
FAB/INLINE long-format (lot_id-wafer_id-time-item_id-value)	
ET hive or flat	(flow 가 둘 다 지원)
원자성 (쓰는 도중 절대 안 보임)	
폴더 규약 FAB/... vs 1.RAWDATA_DB_FAB/...	prefix 정렬 필요

연결 옵션

A. 공유 워크스페이스 직접 쓰기 (가장 단순)

```
# settings.json
"s3": {
  "endpoint_url": "",
  "bucket": "db",
  "prefix": "",
  "fake_local_path": "/config/work/sharedworkspace/DB"
}
# executor.py:196 -
s3_key = f"1.RAWDATA_DB_{plan.source}/{plan.product}/date={plan.date}/part-0.parquet"
```

→ Valve 가

/config/work/sharedworkspace/DB/db/1.RAWDATA_DB_FAB/PRODA/date=.../part-0.parquet 에
쓰고, flow 의 db_root 를 /config/work/sharedworkspace/DB/db 로 맞추면 바로 인식.

B. 실 S3 경유 (사내 프로덕션) — Valve 가 s3 에 올리고, flow 의 s3_ingest 가 다운싱크해서 db_root 로 풀어놓는
방식. 각 앱이 S3 를 공유하되 파일시스템은 분리.

양방향 원인 체인 (에이전트 orchestrator 경유)

```
# Valve      →      → flow
executor.py failure → ops.dispatch_alert
→ POST settings.alerts.flow_notify_url ( : http://flow/api/valve/alert)
→ flow      notice      →      belltray

# flow      "      "      → orchestrator  Valve
flow      → orchestrator → GET valve/api/agent/diagnose
→ POST valve/api/agent/apply-fix (retry_chunk, dry_run=false)
→ Valve      → flow
```

16. 트러블슈팅 플레이북

증상	원인 1순위	해결
모든 chunk 가 HY000 재시도 후 실패	과내 API 키 미등록 또는 만료	Settings > Lake API > api_key 갱신. Logs 탭에서 error 메시지 확인
partition 이 항상 completeness_failed	tolerance_pct 가 너무 타이트	Settings > 스케줄 > tolerance_pct 를 0.5 → 1.0 로.

증상	원인 1순위	해결
특정 소스만 반복 timeout_reshard	probe 가 부정확 → chunk 가 과대	제품 탭 → 해당 소스 → probe skip 체크. 또는 shard_1
S3 업로드 실패만 쌓임	네트워크 장애 or 권한	Settings > S3 credentials · endpoint_url 확인. upload_mode=int
기동 시 config_s3_unreachable S3 업로드 실패 (정상 동작 — 로컬 fallback)	하버드 재 기동이라 S3 에 아직 설정 파일 없는 경우. UI 에서 설정	
알람이 너무 자주 와서 피로	rate_limit·dedupe 미설정	Settings > 알림 > max_per_hour=60 · dedupe_window_sec=60 권
Valve 재기동 시 10만+ 플랜 로딩 느림	plans.jsonl 이 커짐	자동 rotate (50MB 기본) 가 snapshot 라인 남겨 1줄 복원 — 정

17. API 레퍼런스 (자주 쓰는 것만)

Jobs

메소드	엔드포인트	역할
GET	/api/jobs/state	현재 snapshot (plans · chunks · partitions)
GET	/api/jobs/stream	SSE — 실시간 이벤트
POST	/api/jobs/enqueue	단일 (제품,소스,날짜) 실행
POST	/api/jobs/enqueue-all	backfill 창 전체 일괄
POST	/api/jobs/enqueue-product	제품 단위 초기 시딩
POST	/api/jobs/retry-partition	(제품,소스,날짜) 재실행
POST	/api/jobs/cancel	chunk 취소
POST	/api/jobs/probe-invalidate	probe 캐시 무효화
GET	/api/jobs/s3-pending	S3 업로드 대기 큐 조회
POST	/api/jobs/s3-flush	대기 큐 즉시 플러시
GET	/api/jobs/history	실행 이력 (필터 가능)

Schedule · Products · Source Types

메소드	엔드포인트	역할
GET	/api/schedule	스케줄 items
GET	/api/schedule/products	products 조회
POST	/api/schedule/products	products 저장 (자동 마이그레이션)
GET	/api/schedule/source-types	소스 타입 레지스트리
POST	/api/schedule/source-types	소스 타입 저장
GET	/api/schedule/columns	제품/소스 컬럼 풀

Ops · Agent · Alerts

메소드	엔드포인트	역할
GET	/api/metrics	JSON 메트릭 (chunk/partition status, p50/p95)
GET	/api/metrics/prom	Prometheus text format
POST	/api/alerts/test	webhook 연결 테스트
GET	/api/alerts/recent	최근 200건 알람 버퍼
GET	/api/agent/diagnose	현재 이상 목록
POST	/api/agent/suggest-fix	룰 기반 fix 제안
POST	/api/agent/apply-fix	액션 실행 (dry_run 우선)
GET	/api/agent/actions	화이트리스트 액션 카탈로그
GET	/api/agent/audit	agent 호출 감사 로그

이 매뉴얼의 pdf 생성 스크립트: Valve/scripts/gen_manual_pdf.py. 섹션 변경 후 재생성해 docs/valve_manual.pdf 로 덮어쓰기.

문의·개선 제안: github.com/goood2280/valve/issues.